



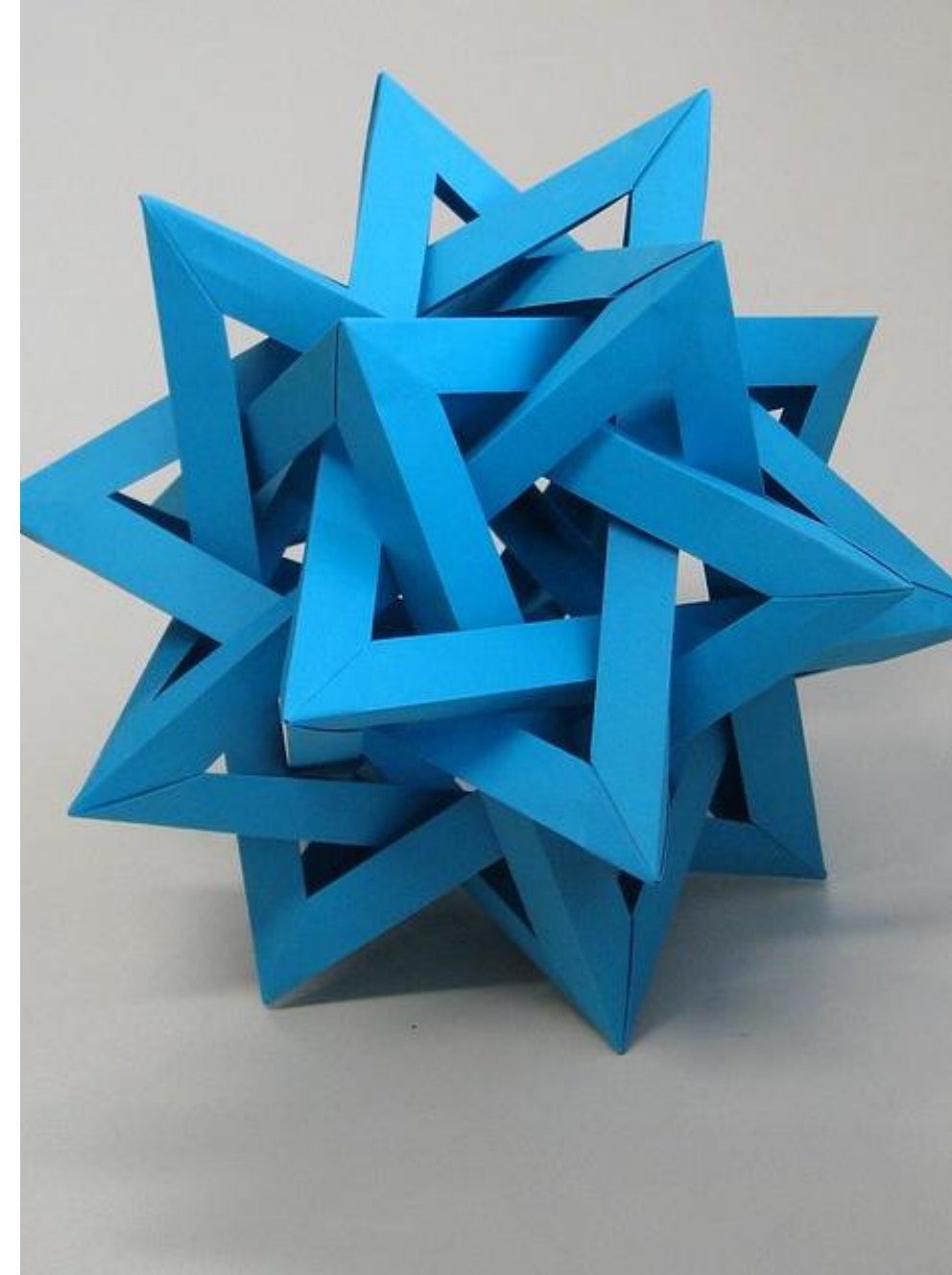
Politecnico
di Torino

Dipartimento
di Automatica e Informatica

14BHDxx Informatica

ING. MEC/ELT/INF/CHI/MTM – CORSO #9 (DIP-INS)

PROF. FULVIO CORNO



[This Photo](#) by Unknown Author is licensed under [CC BY-SA](#)

Welcome

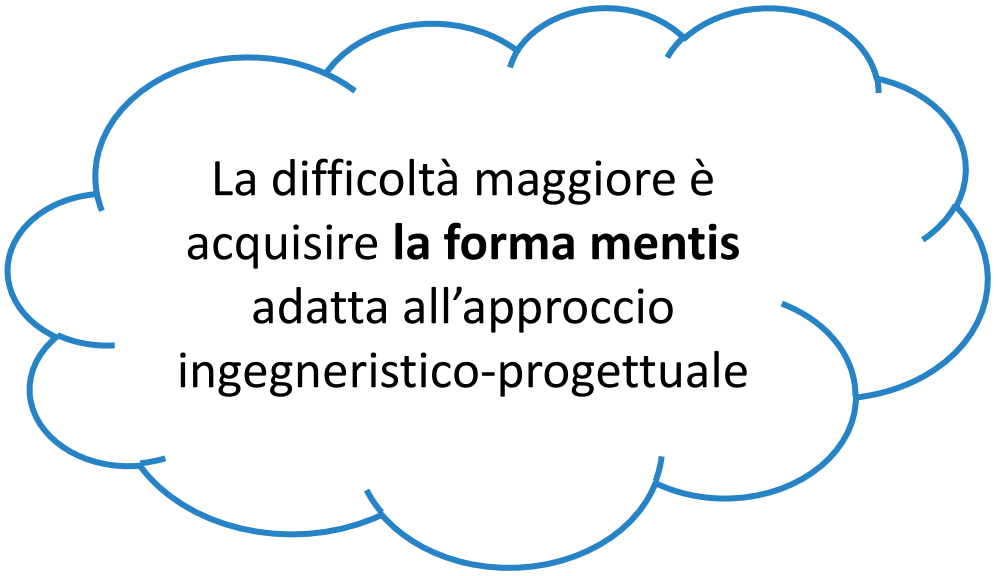
Corso di Informatica (14BHD)

- Insegnamento obbligatorio, 8 crediti, 1° Semestre
- Corso n. 9
 - Studenti con cognomi compresi tra DIP e INS
 - Iscritti ai corsi di ing. Meccanica, Elettrica, Informatica, Chimica, Matematica
- Docenti:
 - **Fulvio Corno** (lezioni, esercitazioni in aula)
 - **Roberta Bardini** (lezioni, esercitazioni in aula)
 - **Lorenzo Martini** (esercitazioni in laboratorio)
 - **Giulia Masi** (esercitazioni in laboratorio)



Benvenuti ad Ingegneria

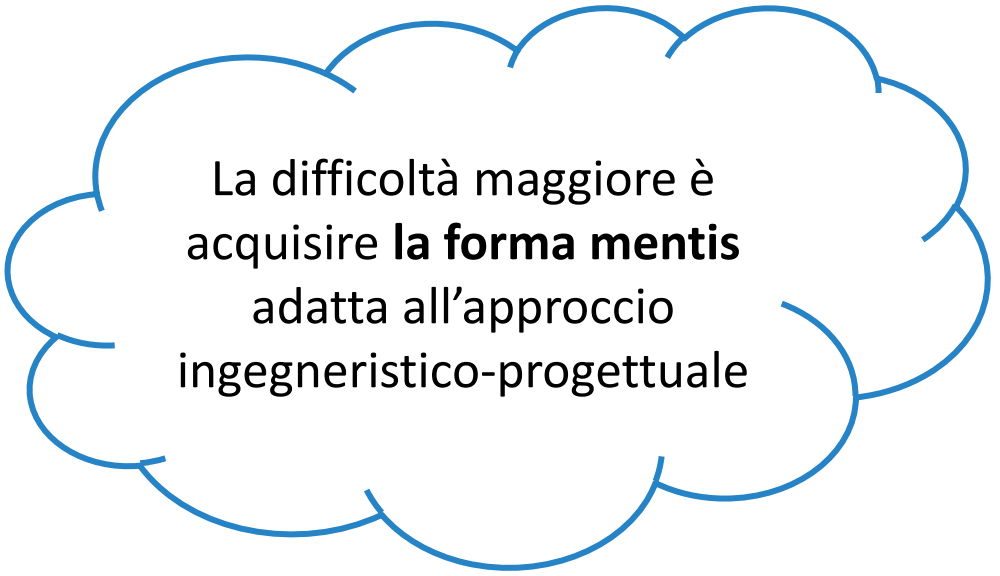
- Questo è il primo corso di Ingegneria che affronterete
- Ingegneria =
 - Saper progettare
 - Risolvere problemi
 - Trovare soluzioni
 - Soddisfare le specifiche
 - Nel rispetto dei vincoli
 - Con gli strumenti disponibili



La difficoltà maggiore è
acquisire **la forma mentis**
adatta all'approccio
ingegneristico-progettuale

Benvenuti ad Ingegneria

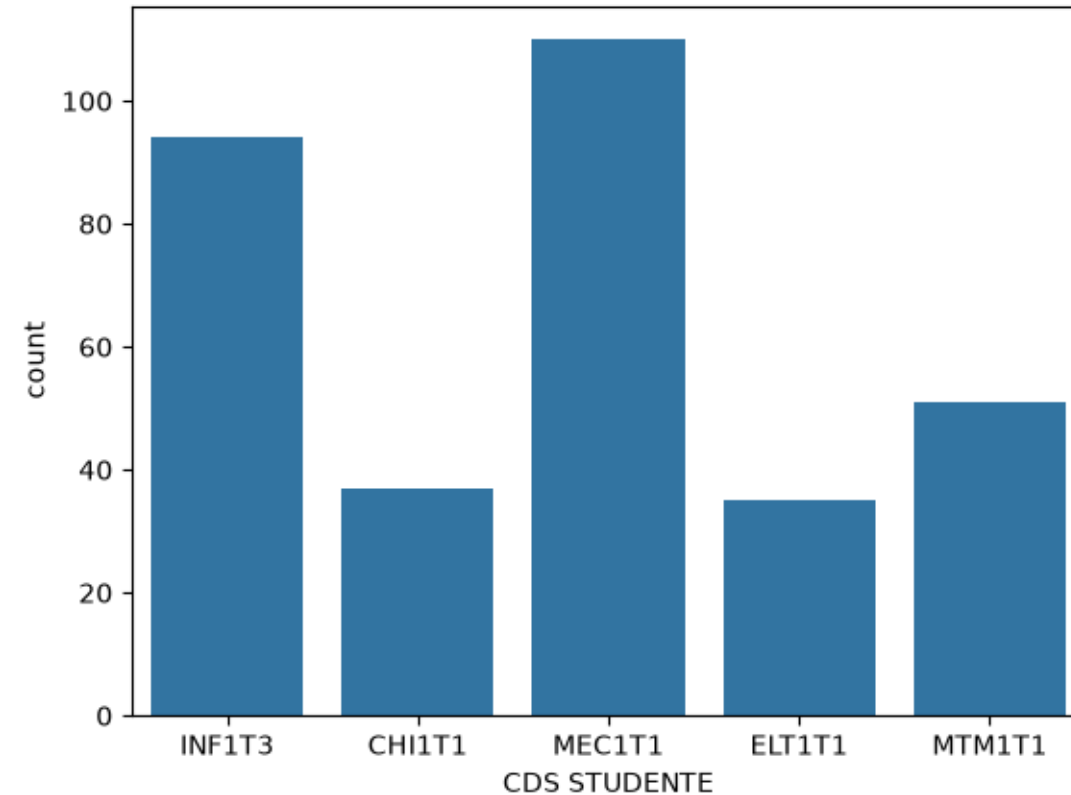
- Questo è il primo corso di Ingegneria che affronterete
- Ingegneria =
 - Saper progettare
 - Risolvere problemi
 - Trovare soluzioni
 - Soddisfare le specifiche
 - Nel rispetto dei vincoli
 - Con gli strumenti disponibili
- Ingegneria informatica =
 - Problemi di ogni genere (calcolo, gestione dati, interazione, ...)
 - Lo strumento è il calcolatore



La difficoltà maggiore è
acquisire **la forma mentis**
adatta all'approccio
ingegneristico-progettuale

Chi siamo?

```
import pandas as pd
import seaborn as sns
studenti = pd.read_csv('14BHDWD_2027.csv')
sns.countplot(data=studenti, x='CDS STUDENTE')
```



Chi siamo? – Instant Polling



Linguaggi di programmazione

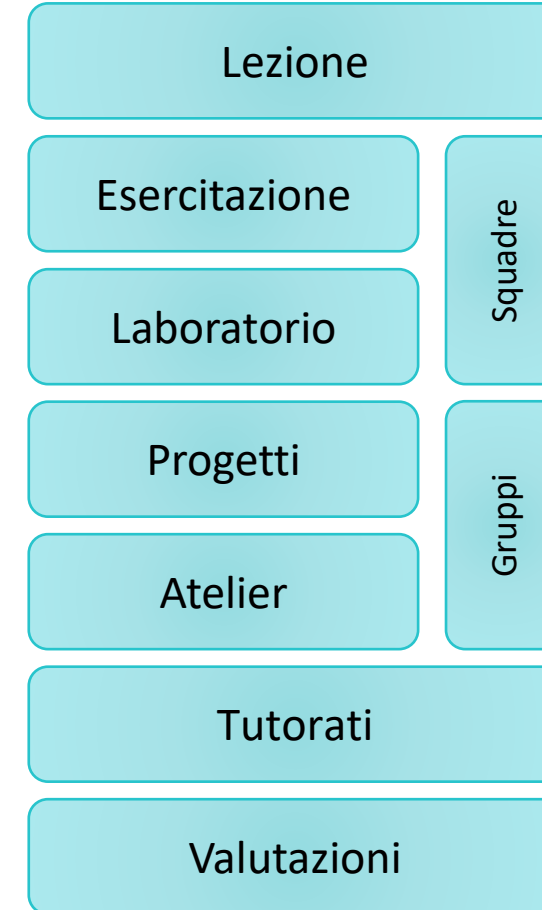
- In questo corso **si parte da zero**, non è richiesto conoscere già un linguaggio
- Le capacità di **problem solving** si trasferiscono facilmente da un linguaggio all'altro
-  Talvolta, dovrete dis-imparare alcune cattive abitudini apprese in precedenza

Premessa – Studiare al Poli

La didattica al Politecnico di Torino

- Un **insegnamento** : tante tipologie didattiche

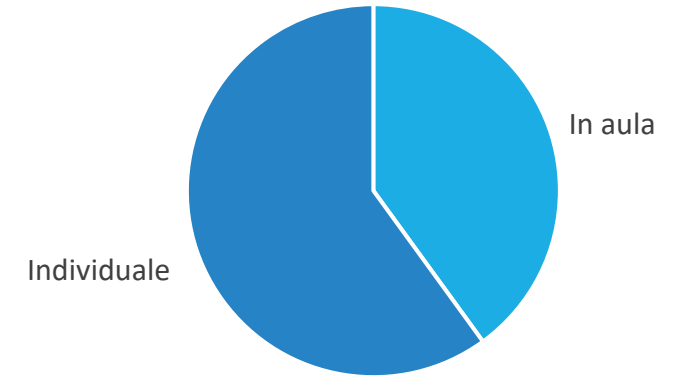
- Per l'apprendimento
- Per il consolidamento
- Per l'approfondimento
- Per l'autovalutazione
- Per il recupero
- Individuali o in gruppo



La didattica al Politecnico di Torino

- Un **insegnamento** : Crediti Formativi Universitari (**CFU**)

- Misura dell'impegno complessivo per studente
- 1 CFU = 25 ore di lavoro
- di cui 10 ore in aula (in orario)
- di cui 15 ore di impegno individuale



CFU	Ore	In aula	Individuali
6	150	60	90
8	200	80	120
10	250	100	150
12	300	120	180

Durante il semestre
+
Preparazione all'esame

Raccomandazioni

Partecipare attivamente e con continuità alle attività **proposte**

Dedicare **tempo** allo studio partendo dalle prime settimane. Stare al passo.



Sfruttare i momenti di autovalutazione e **tutorato**.
Interagire con i docenti.

Non 'abbandonare' nessun insegnamento. Puntare a sostenere **tutti** gli esami

Raccomandazioni

Partecipare attivamente e con continuità alle attività proposte.

Dedicare **tempo** allo studio partendo dalle prime lezioni e al passo.

Se incontrate difficoltà nello studio o nella vita universitaria, informatevi sui molti Servizi di **Accoglienza, inclusione e sostegno**

<https://www.polito.it/didattica/servizi-e-vita-al-politecnico/accoglienza-inclusione-e-sostegno>

Sfruttare i momenti di autovalutazione e **tutorato**.
Interagire con i docenti.

Non ‘abbandonare’ nessun insegnamento. Puntare a sostenere **tutti** gli esami

Cos'è questo corso?

Link utili

- Sito del corso (ufficiale):
 - <https://elite.polito.it/> → Teaching → Informatica (14BHD)
 - Link breve: <https://elite.polito.it/info>
- Materiale, laboratori, esercizi
 - <https://github.com/polito-informatica>
- Gruppo Telegram
 - <https://t.me/politoinfo2627> →
- Canale Telegram per **tutorati**
 - <https://t.me/+QttiF2nn0hFiNDg0>



Programma dell'insegnamento

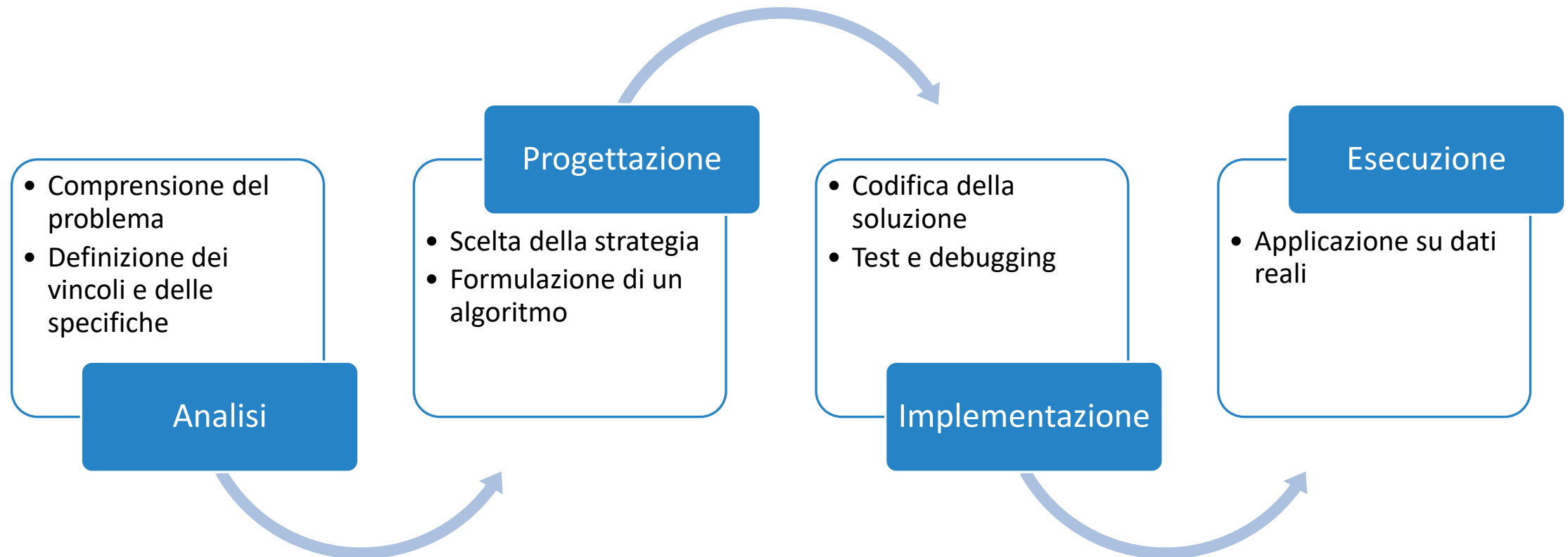
- Metodologie di *Problem Posing and Solving* (PPS)
 - Tecniche di analisi basate su flow-chart e pseudo-code
 - Implementazione attraverso programmi informatici
- Strutture dati e rappresentazione dell'informazione nel PPS
 - Numeri, Stringhe, Vettori, Sequenze, Liste, Insiemi, Dizionari, ...
- Linguaggio di programmazione Python
 - Maggior semplicità sintattica e maggior potenza espressiva
 - Possibilità di affrontare esercizi con uno scopo applicativo più diretto
 - Ambiente di lavoro adeguato ai sistemi operativi moderni
 - Disponibilità di numerose librerie adatte a diversi campi applicativi (che potranno essere introdotte negli insegnamenti successivi).

Contenuti

- Teoria (9h)
 - Cenni di Informatica generale ed impatti dell'informatica e del digitale
 - Struttura ed architettura del calcolatore, linguaggi, applicazioni
 - Rappresentazione dell'Informazione
- Problem Solving (12h)
 - Approccio alla logica dei problemi
 - I passaggi del processo di Problem Solving
 - Tipologie di problemi e di approcci risolutivi
- Programmazione Python (41h)
 - Numeri e Stringhe
 - Decisioni
 - Cicli
 - Liste (vettori)
 - Insiemi e dizionari (array associativi)
 - File
 - Funzioni
- Laboratori (12 x 1,5 = 18h)

...e cioè cosa impariamo a fare?

- Quali sono i nomi più frequenti in quest'aula?



Una possibile soluzione... in Python

```
import csv
from matplotlib import pyplot

# Leggi l'elenco degli studenti e salvalo in un'array
def leggi(nome_file):
    file = open(nome_file, 'r')
    reader = csv.reader(file)
    prima = True
    studenti = []
    for line in reader:
        if prima: # skip first line (headers)
            prima = False
        else:
            studenti.append(line)
    file.close()
    return studenti

# estrai i nomi di battesimo da un elenco di studenti
def estrai_nomi(elenco):
    lista_nomi = []
    for riga in elenco:
        lista_nomi.append(riga[2])
    return lista_nomi

# Calcola le frequenze dei vari nomi presenti in un array
def frequenze(tokens):
    freq = {}
    for token in tokens:
        if token in freq:
            freq[token] = freq[token] + 1
        else:
            freq[token] = 1
    return freq
```

```
# calcola il massimo valore presente nelle frequenze
def max_frequenza(freq):
    return max(freq.values())

def nomi_piu_frequenti(freq, max):
    return [nome for (nome, frequenza) in freq.items() if frequenza == max]

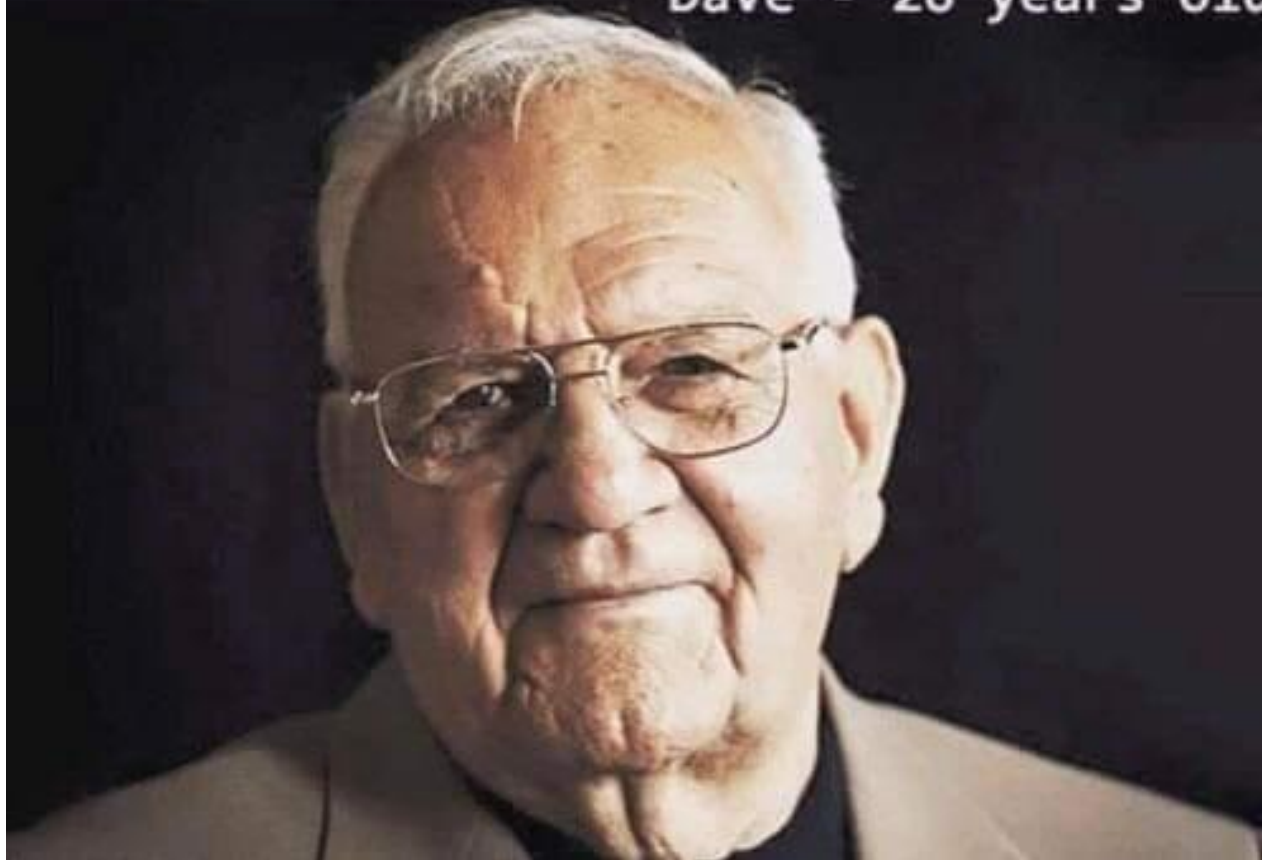
FILENAME = '01TXYOYV_2020.csv'
def main():
    stud = leggi(FILENAME)
    nomi = estrai_nomi(stud)
    print(f"Nella classe ci sono {len(stud)} studenti")
    freq = frequenze(nomi)
    max_freq = max_frequenza(freq)
    print(f"Il nome più frequente compare {max_freq} volte")
    nomi_max = nomi_piu_frequenti(freq, max_freq)
    print(f"Si tratta di : {nomi_max}")
    # estrai solo i nomi che compaiono almeno 3 volte
    freq2 = {k: v for (k, v) in freq.items() if v >= 3}
    print(
        f"I nomi che compaiono più volte sono {'', '.join(sorted(list(freq2.keys()))))."
    )

    pyplot.barh(list(freq2.keys()), freq2.values())
    pyplot.show()

main()
```

“Programming is
not stressful at all”

Dave - 26 years old

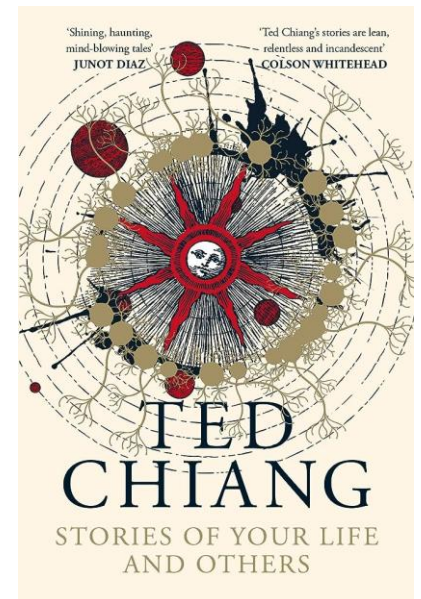




“What I had not realized is that extremely short exposures to a relatively simple computer program could induce powerful delusional thinking in quite normal people.”

— Joseph Weizenbaum

A cosa serve imparare a programmare?



Il Corso di Informatica in breve

Seguire le lezioni con
il **cervello acceso**

Anche il **computer**
acceso, se volete

Interagire, provare,
fare domande

Compagni, Docenti,
Tutori, Assistenti,
Telegram, ...

Iniziare da subito!

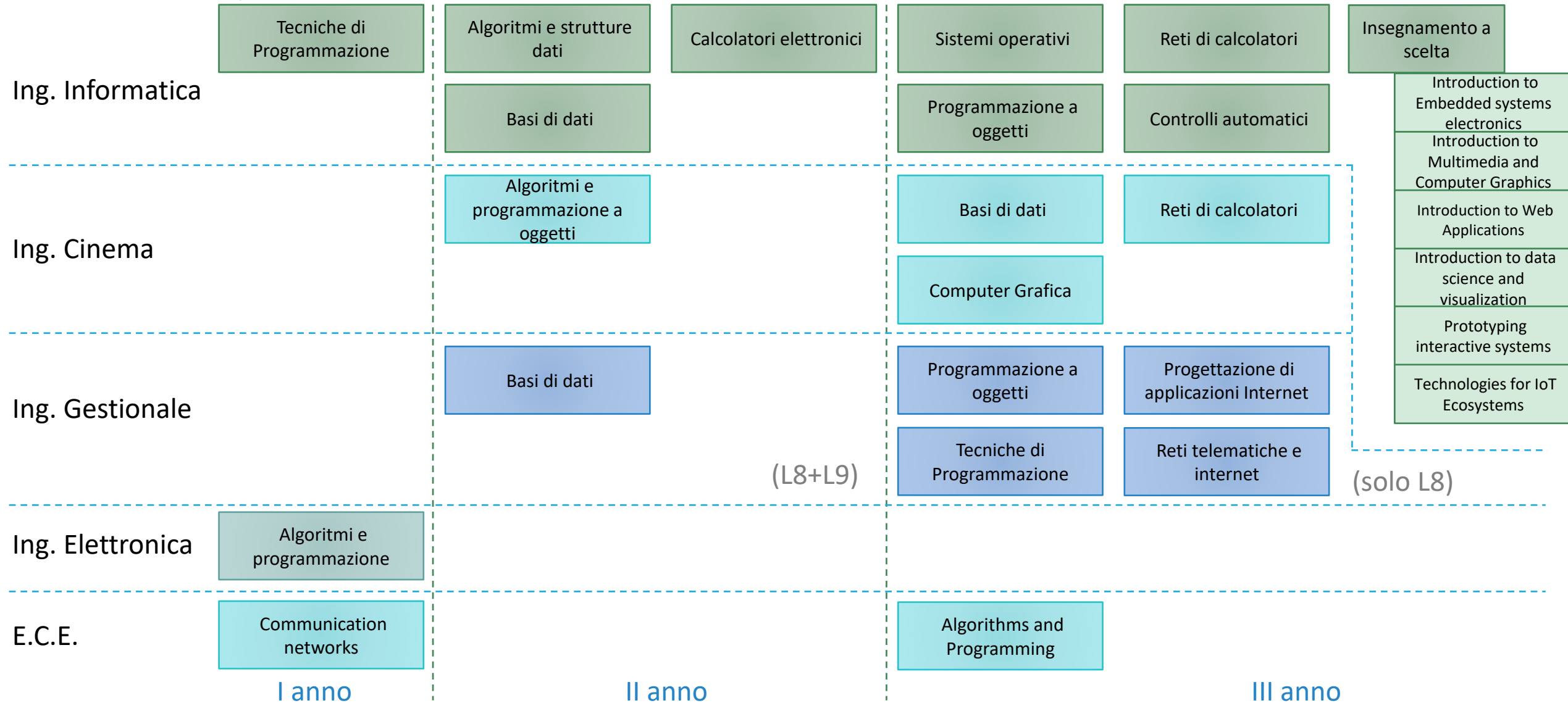
Partecipare ai
laboratori

Fare gli esercizi
proposti (tutti)

Sfruttare i tutorati

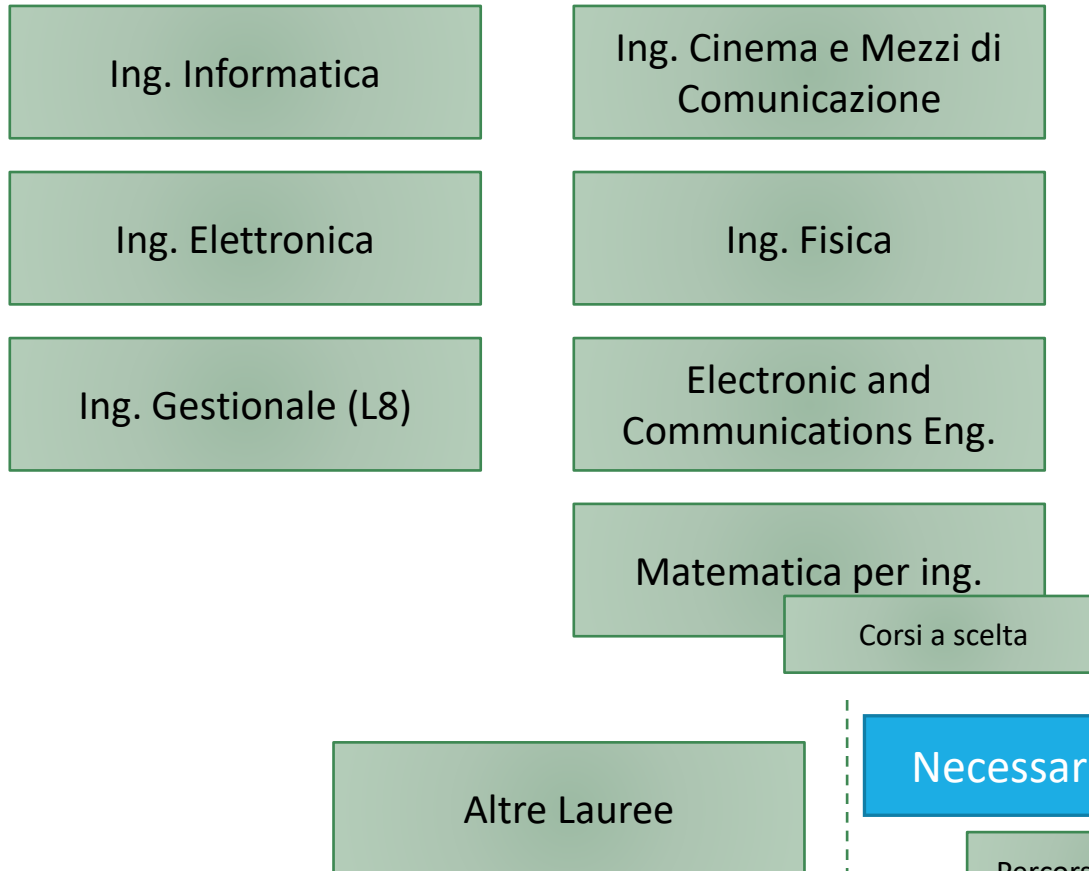
In modo attivo

Dopo «Informatica»

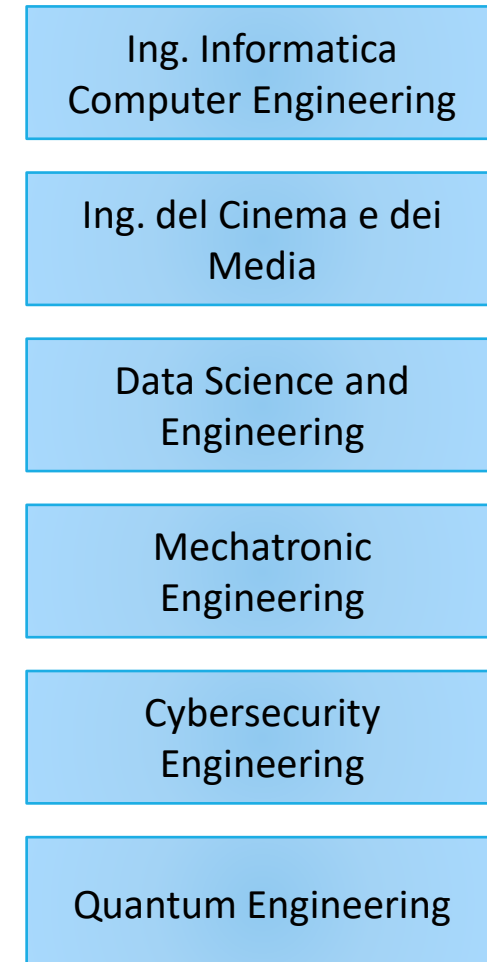


Dopo «Dopo «Informatica»»

Laurea



Laurea Magistrale



Organizzazione del corso

Sito del corso

Tutto il materiale sarà disponibile su questo sito

- Slide
- Laboratori
- Esempi svolti
- Video Lezioni
- Temi d'esame
- Calendario lezioni
- ...

<https://elite.polito.it/info>

The screenshot shows the e-Lite website interface. At the top, there is a navigation bar with links: e-Lite, News, People, Research, Teaching, and Thesis. A sidebar on the left lists various courses, with 'Informatica (14BHD)' selected and highlighted. The main content area displays the course title 'Informatica (14BHD)' and a brief description: 'Pagina ufficiale del corso di Informatica (codice 14BHD, (DIQ - JZZ - Corso 3), anno accademico 2023/2024) presso il Politecnico di Torino.' Below this, there are six content blocks arranged in a 3x2 grid: 'Introduzione' (Avvisi e informazioni di base), 'Materiale di studio' (Lucidi, videolezioni, esercizi, ...), 'Calendario delle lezioni' (Calendario delle lezioni e materiale utilizzato in ciascuna lezione), 'Laboratorio' (Testi e soluzioni delle esercitazioni di laboratorio), 'Esame' (Materiale per la preparazione all'esame), and a 'Previous < Exam' button. At the bottom right, there is a 'Next Introduzione >' button. A double arrow icon is visible at the bottom left of the sidebar.

Comunicazioni



- **Tutti** i contatti con i docenti avverranno sulla piattaforma **Telegram**
 - Non inviare e-mail ai docenti, ma utilizzare il **gruppo Telegram**
 - Non inviare messaggi privati, se non richiesti dai docenti stessi
- **Iscriversi (obbligatorio!) all'indirizzo**
<https://t.me/politoinfo2627>



Orario Settimanale

Alcune lezioni si svolgeranno
on-line o tramite video
registrazioni

	Lunedì	Martedì	Mercoledì	Giovedì		Venerdì
08:30-10:00						
10:00-11:30						
11:30-13:00			Lezione / Esercitazione Aula 2	Laboratorio Squadra 1 Aula 3D		
13:00-14:30			Lezione / Esercitazione Aula 2	Laboratori o Squadra 2 Aula 3D	Laboratori o Squadra 3 Aula 5D	
14:30-16:00						
16:00-17:30						
17:30-19:00	Lezione / Esercitazione Aula 2					

Struttura del corso

- Programmazione e Python : 41 ore
- Teoria: 9 ore
- Problem solving : 12 ore
- Laboratorio : 18 ore (x 3 squadre)
- Tutorato settimanale

- Totale: 80 ore/studente in aula
 - + circa 120 ore di studio individuale

Laboratori

- La parte più importante del corso, in cui imparare a risolvere problemi e scrivere programmi
- Il corso è diviso in 3 squadre
 - Presso i Laboratori Informatici (LAIB)
- Inizio laboratori: **01/10/2026**
- Testo pubblicato sul sito del corso
- È necessario installare il software richiesto (*Python* e *Visual Studio Code*) - vedere le istruzioni di installazione sul portale

Suddivisione in squadre

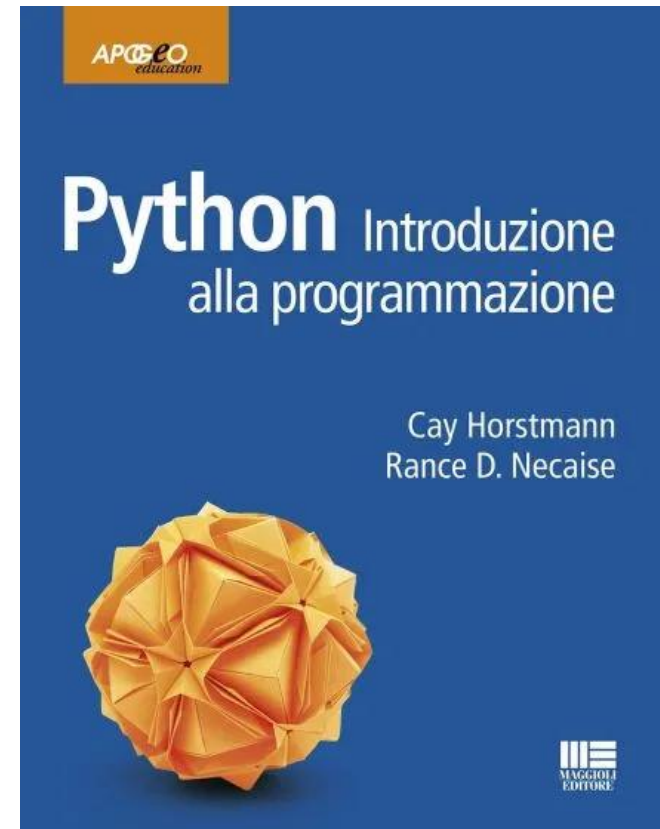
- Squadra 1 (giovedì 11:30, Aula 3D): xxxxx
- Squadra 2 (giovedì 13:00, Aula 3D): xxxxx
- Squadra 3 (giovedì 13:00, Aula 5D): xxxxx

Libri di testo

TESTO COMPLETO IN ITALIANO



VERSIONE RIDOTTA (CAPITOLI 1-8)
CORRISPONDENTE AL PROGRAMMA SVOLTO



Libri di testo: Informazioni dettagliate

TESTO COMPLETO IN ITALIANO

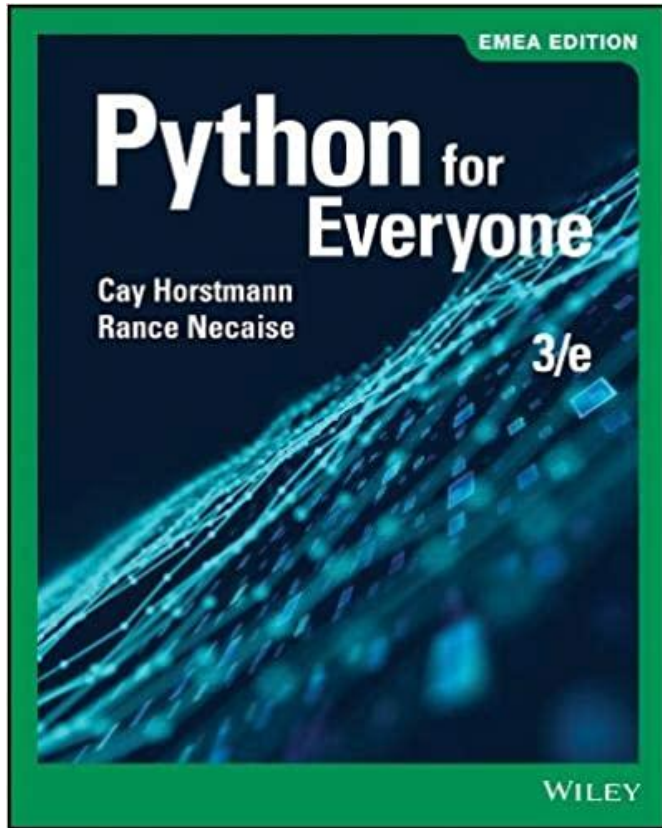
- Concetti di informatica e fondamenti di Python
- Seconda edizione - Giugno 2019 (II° Edizione)
- Cay Horstmann - Rance D. Necaise
- Maggioli Editore
- Giugno 2019
- ISBN 978-8891635433
- <https://www.maggiolieditore.it/concetti-di-informatica-e-fondamenti-di-python.html>

VERSIONE RIDOTTA (CAPITOLI 1-8) CORRISPONDENTE AL PROGRAMMA SVOLTO

- Python - Introduzione alla programmazione
- Versione ridotta della Seconda Edizione
- Cay S. Horstmann, Rance D. Necaise
- Maggioli Editore
- Luglio 2023
- ISBN: 978-8891663979
- <https://www.maggiolieditore.it/python-introduzione-alla-programmazione.html>

Libri di testo

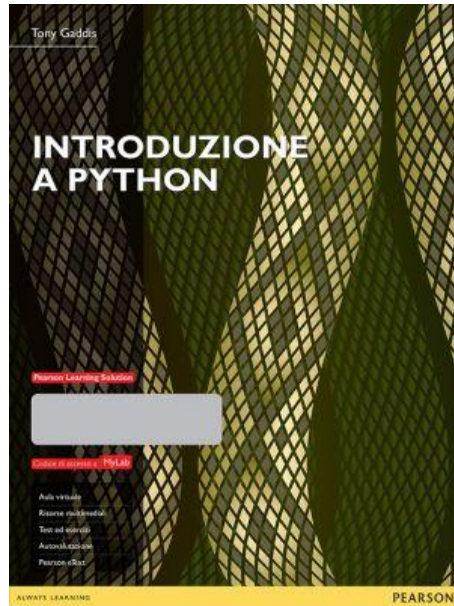
TESTO ORIGINALE IN LINGUA INGLESE



INFO DETTAGLIATE

- Python For Everyone
- 3rd Edition
- Cay S. Horstmann, Rance D. Necaise
- Wiley
- ISBN: 978-1-119-49853-7 December 2018
- <https://www.wiley.com/en-it/Python+For+Everyone,+3rd+Edition-p-9781119498537>

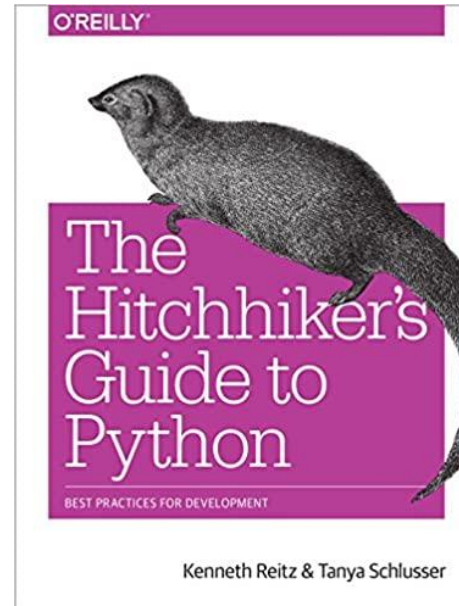
Altre risorse...



Introduzione a Python
Tony Gaddis
Pearson - ISBN: 9788891900999



Introduzione a Python per
l'informatica e la data science
Paul J. Deitel, Harvey M. Deitel,
Pietro Codara, Carlo Mereghetti
Pearson - ISBN: 9788891915924



The Hitchhiker's Guide to Python:
Best Practices for Development
Kenneth Reitz, Tanya Schlusser
O'Reilly Media - ISBN-13: 978-
1491933176

Gratis su:

<https://docs.python-guide.org/>

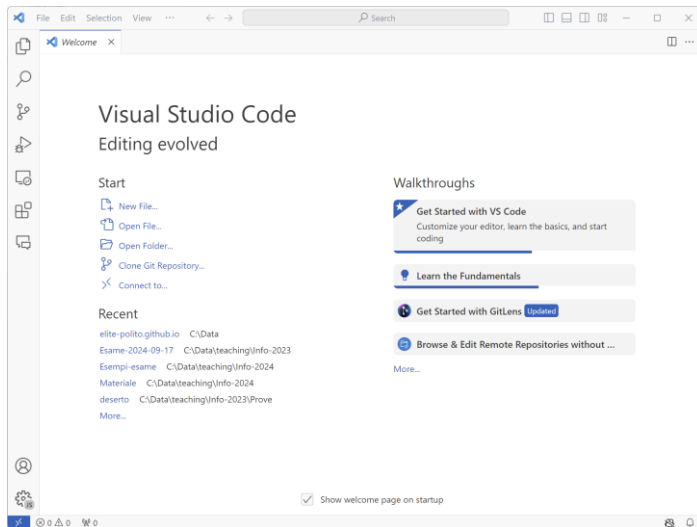


<https://www.python.org/>
<https://docs.python.org/3/>
<https://docs.python.org/3/tutorial/>

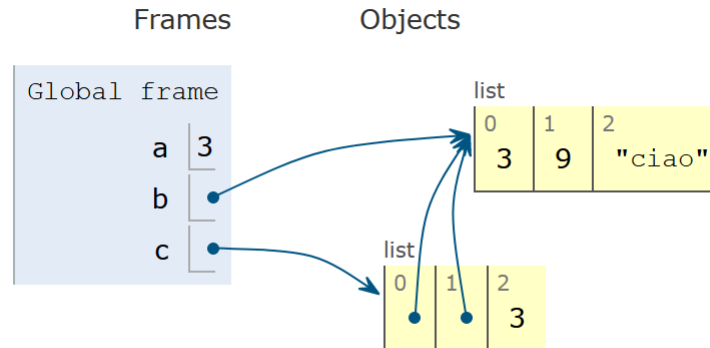


<https://realpython.com/>

Strumenti per programmare



<https://code.visualstudio.com/>



<http://pythontutor.com>



<https://www.jetbrains.com/pycharm/>

- Professional (gratis per docenti e studenti)
- Community (gratis per tutti)

Istruzioni di installazione

148HDxx - Informatica

Installazione software (piattaforma Windows)

[Sommario](#)

FASE A: Installare l'interprete Python	2
Gli ambienti di sviluppo PyCharm	4
FASE B: Installazione di PyCharm Edu (opzione consigliata)	4
FASE C: Attivazione di PyCharm Edu	7
FASE D: Creazione di un nuovo progetto in PyCharm Edu	8
In alternativa (opzione avanzata): PyCharm Community o Professional	10
FASE B: Installazione di PyCharm Community o Professional	10
FASE C: Attivazione di PyCharm Community/Professional e della relativa licenza d'uso	12
FASE D: Creazione di un nuovo progetto in PyCharm Community/Professional	14

128HDxx - Informatica

Installazione software (piattaforma macOS)

[Sommario](#)

FASE A: Installare l'interprete Python	2
FASE B: Installare l'ambiente di sviluppo PyCharm	4
FASE C: Attivazione di PyCharm e della relativa licenza d'uso	5
FASE D: Creazione di un nuovo progetto in PyCharm	8

<http://bit.ly/polito-informatica>

Informazioni pratiche

Tutorati

- Nel Corso del semestre saranno disponibili dei *tutor* (in presenza e on-line) per aiutarvi a rivedere *esercizi* e chiarire dubbi e *difficoltà*
- I tutorati sono un'opportunità da sfruttare!
- Per info sui tutorati ed altri avvisi generali unirsi al canale Telegram:
 - <https://t.me/+QttiF2nn0hFiNDg0>

Tutorato

- Cosa è il tutorato?
 - Spazio di supporto attivo, non una lezione extra.
 - I tutor aiutano a chiarire dubbi, errori negli esercizi e concetti poco chiari.
- Come funziona?
 - Modello bottom-up: porti dubbi, esercizi, errori → il tutor ti aiuta a risolverli.
 - Attività ex-cathedra ridotte al minimo.
 - In caso di “stallo”, possono essere proposti piccoli esercizi-milestone.
- Obiettivo:
 - 👉 Aiutare gli studenti a diventare attivi e autonomi nello studio.



Tutti i dettagli nel Vademecum (nel materiale del corso)

Esame

Contenuti dell'esame

- 3 domande brevi sulla parte teorica del corso (6 punti totali, minimo 3 punti per superare l'esame)
- Un esercizio di programmazione (26 punti)
 - Con la possibilità di usare uno strumento di sviluppo per la scrittura del codice
 - Sarà consegnato il codice sorgente del programma sviluppato
 - Il codice verrà corretto manualmente (valuteremo la qualità della soluzione, e il rispetto delle richieste del problema, in maniera indipendente da eventuali errori sintattici e dall'effettivo funzionamento del programma)
- Nelle ultime settimane del corso vi proporremo diversi esercizi di simulazione dell'esame, in modo da poter familiarizzare con le modalità di esame e con le conoscenze richieste per passarlo
- Temi d'esame: <https://github.com/polito-informatica/Esempi-esame>

Cosa serve per passare [bene] l'esame?

- Capacità logico-razionali di analisi e di sintesi
 - Comprendere i propri processi risolutivi e saperli formalizzare
- **Svolgere tutti gli esercizi** proposti
 - Davvero
 - Anche quelli [che sembrano] facili
 - Da soli
 - Su Personal Computer
 - Verificarli con dati diversi
 - Cercare di metterli in crisi
- Inventarsi nuovi problemi, o varianti di quelli proposti
 - E poi risolverli
- Sfruttare le ore di **Tutorato**

Accertamenti intermedi – perché?

- Mi aiuta a seguire con continuità durante il semestre
- Mi aiuta a non scoprire all'ultimo momento le difficoltà del corso
- Mi aiuta ad auto-valutarmi nella comprensione e nella messa in pratica degli argomenti
- Mi stimola a farmi domande ed a fare domande ai docenti
- Mi accompagna all'esame essendo sicuro/a di essere preparato/a
- Mi permette di ottenere qualche punto in più per l'esame stesso

Accertamenti intermedi – cosa?

- 1 accertamento intermedio di Teoria
 - 3 domande, identiche a quelle dell'esame
 - 2 punti per domanda, totale 6 punti
 - se punteggio ≥ 3 , è possibile utilizzare tale punteggio all'esame, senza quindi dover rispondere alle domande di Teoria (= Risparmio tempo per la programmazione)
- 3 accertamenti intermedi di Programmazione
 - 1 punto per ciascun accertamento
 - brevi domandine di programmazione
 - Necessaria partecipazione almeno a 2 prove su 3
 - I punti saranno sommati ("bonus") agli esami con punteggio ≥ 18
- Punteggi validi per tutti gli appelli del 2026/27 (anche più volte)

Accertamenti intermedi – quando?

- Sempre nelle ore di laboratorio (No ore extra! No sabati o altro)
- Programmazione 1: 15 Ottobre (Settimana 4)
 - Variabili, numeri, stringhe, condizionali, cicli while e for
- Programmazione 2: 5 novembre (Settimana 7)
 - Liste, tabelle, funzioni
- Programmazione 3: 17 dicembre (Settimana 13)
 - Dizionari, File, Strutture dati complesse
- Teoria: 26 novembre (Settimana 10)

Uno sguardo a Python

VISIONE GENERALE DELL'ECOSISTEMA PYTHON

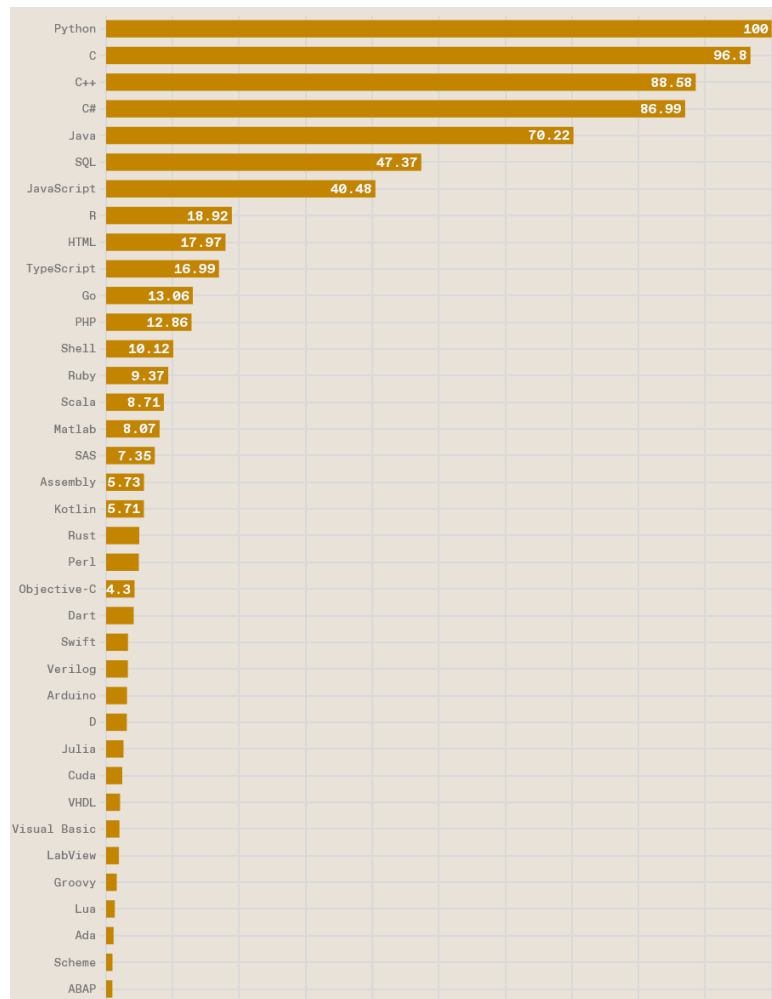
Il linguaggio Python



- Linguaggio gratuito ed open-source
- Disponibile per tutti i sistemi operativi
 - Windows, Mac OS X, Linux
 - Sistemi embedded, Raspberry PI, Android
- Progettato negli anni '90 da Guido Van Rossum
 - Sintassi semplice, pulita, regolare
 - Approccio «batterie incluse»
 - Ampia libreria di funzioni standard
 - Basso gradino d'accesso
 - Linguaggio interpretato
- Sterminata documentazione on-line



Diffusione del linguaggio Python



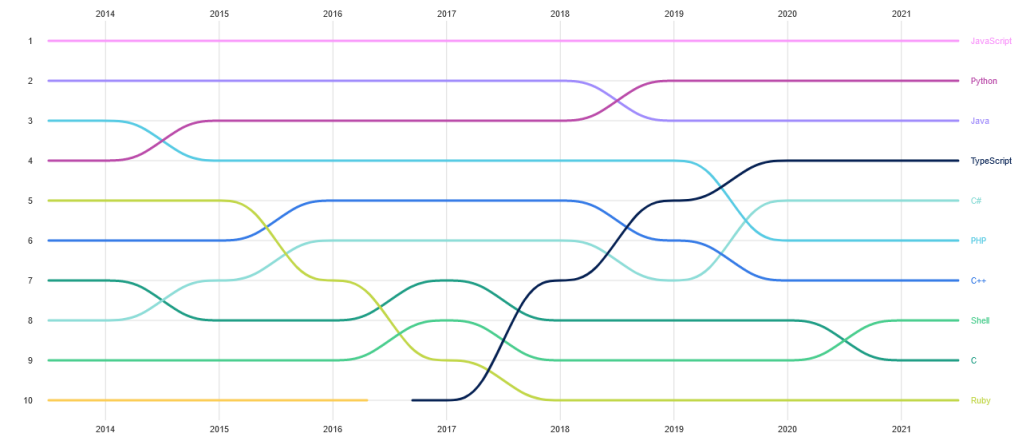
- IEEE Spectrum, 23 Aug 2022
<https://spectrum.ieee.org/top-programming-languages-2022>
 - Top Programming Languages 2022
- Altre statistiche, per i più curiosi:
 - <https://www.tiobe.com/tiobe-index/>
 - <http://pypl.github.io/PYPL.html>
 - <https://octoverse.github.com/>
 - ...

Diffusione del linguaggio Python

Aug 2022	Aug 2021	Change	Programming Language	Ratings	Change
1	2	▲	 Python	15.42%	+3.56%
2	1	▼	 C	14.59%	+2.03%
3	3		 Java	12.40%	+1.96%
4	4		 C++	10.17%	+2.81%
5	5		 C#	5.59%	+0.45%
6	6		 Visual Basic	4.99%	+0.33%
7	7		 JavaScript	2.33%	-0.61%
8	9	▲	 Assembly language	2.17%	+0.14%
9	10	▲	 SQL	1.70%	+0.23%
10	8	▼	 PHP	1.39%	-0.80%
11	16	▲▲	 Swift	1.27%	+0.30%
12	12		 Classic Visual Basic	1.27%	+0.04%
13	22	▲▲	 Delphi/Object Pascal	1.22%	+0.60%
14	23	▲▲	 Objective-C	1.22%	+0.61%
15	18	▲	 Go	0.98%	+0.08%
16	14	▼	 R	0.92%	-0.13%
17	17		 MATLAB	0.90%	-0.08%
18	15	▼	 Ruby	0.82%	-0.18%
19	13	▼	 Fortran	0.81%	-0.32%
20	20		 Perl	0.72%	-0.06%

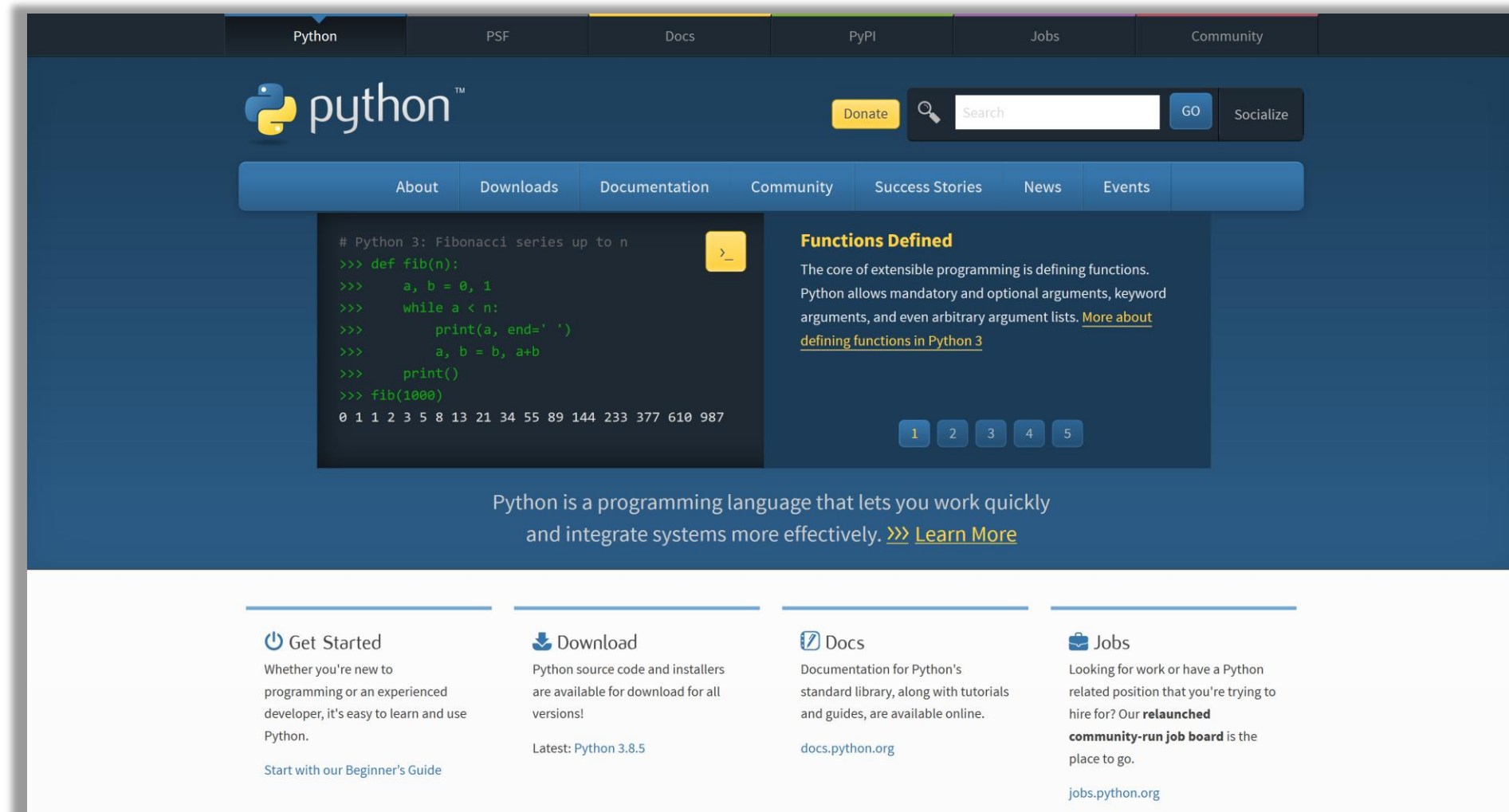
<https://www.tiobe.com/tiobe-index/>

Top languages over the years



<https://octoverse.github.com/>

<https://www.python.org/>



The screenshot shows the Python.org homepage with a dark blue header and a white footer. The header contains navigation links: Python, PSF, Docs, PyPI, Jobs, and Community. The main content area features the Python logo, a search bar, and a 'Donate' button. Below this is a secondary navigation bar with links: About, Downloads, Documentation, Community, Success Stories, News, and Events. The central content area is divided into two columns. The left column displays a code snippet for a Fibonacci function and its output. The right column is titled 'Functions Defined' and explains the core of extensible programming. Below the code snippet is a 'Python is a programming language...' section with a 'Learn More' link. The footer contains four columns: 'Get Started', 'Download', 'Docs', and 'Jobs', each with a brief description and a link.

Python

PSF

Docs

PyPI

Jobs

Community

python™

Donate

Search

GO

Socialize

About

Downloads

Documentation

Community

Success Stories

News

Events

```
# Python 3: Fibonacci series up to n
>>> def fib(n):
>>>     a, b = 0, 1
>>>     while a < n:
>>>         print(a, end=' ')
>>>         a, b = b, a+b
>>>     print()
>>> fib(1000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
```

Functions Defined

The core of extensible programming is defining functions. Python allows mandatory and optional arguments, keyword arguments, and even arbitrary argument lists. [More about defining functions in Python 3](#)

1 2 3 4 5

Python is a programming language that lets you work quickly and integrate systems more effectively. [>>> Learn More](#)

Get Started

Whether you're new to programming or an experienced developer, it's easy to learn and use Python.

[Start with our Beginner's Guide](#)

Download

Python source code and installers are available for download for all versions!

Latest: Python 3.8.5

Docs

Documentation for Python's standard library, along with tutorials and guides, are available online.

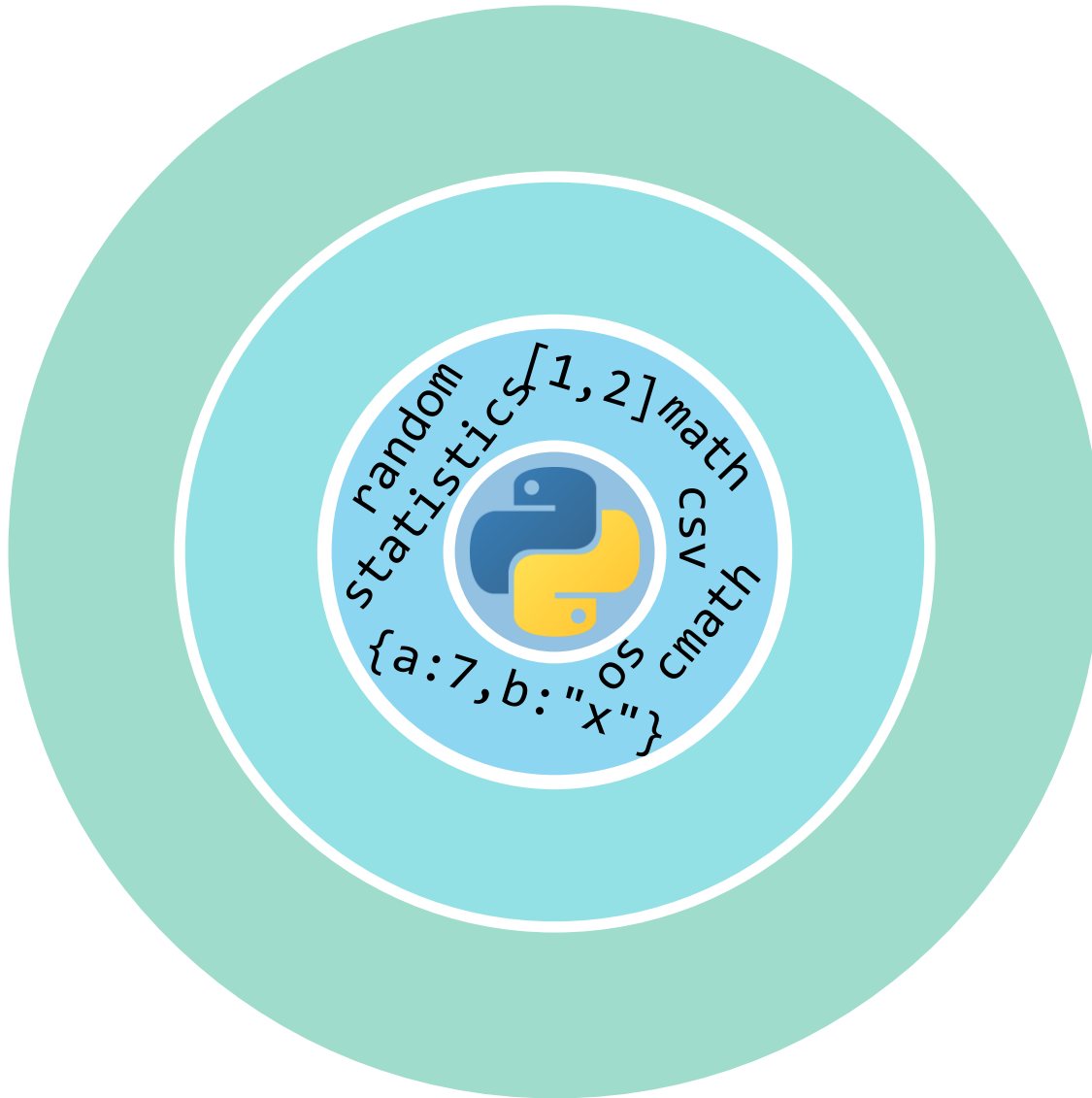
docs.python.org

Jobs

Looking for work or have a Python related position that you're trying to hire for? Our **relaunched community-run job board** is the place to go.

jobs.python.org

Batterie incluse



- **Tipi di dato fondamentali**
 - boolean, int, float, complex, string, regexp
- **Strutture dati fondamentali**
 - liste/array/matrici, tuple, insiemi, dizionari/mappe/hash, file, ...
- **Orientato agli oggetti**
 - Utilizzo semplice e diretto di oggetti predefiniti
 - Possibilità di creare classi ed oggetti personalizzati (avanzato)
- **200+ Moduli nella libreria standard**

200 Moduli della libreria standard

abc	chunk	decimal	getpass	keyword	optparse	queue	sndhdr	telnetlib	unittest
aifc	cmath	difflib	gettext	linecache	os	quopri	socket	tempfile	urllib
argparse	cmd	dis	glob	locale	ossaudiodev (Linux, FreeBSD)	random	socketserver	termios (Unix)	uu
array	codecs	distutils	graphlib	logging	parser	re	spwd (Unix)	test	uuid
ast	codeop	doctest	grp (Unix)	lzma	pathlib	readline (Unix)	sqlite3	textwrap	venv
asynchat	collections	email	gzip	mailbox	pdb	reprlib	ssl	threading	warnings
asyncio	colorsys	encodings	hashlib	mailcap	pickle	resource (Unix)	stat	time	wave
asyncore	compileall	ensurepip	heapq	marshal	pickletools	rlcompleter	statistics	timeit	weakref
atexit	configparser	enum	hmac	math	pipes (Unix)	runpy	string	tkinter	webbrowser
audioop	contextlib	errno	html	mimetypes	pkgutil	sched	stringprep	token	winreg (Win)
base64	contextvars	faulthandler	http	mmap	platform	secrets	struct	tokenize	winsound (Win)
bdb	copy	fcntl (Unix)	imaplib	modulefinder	plistlib	select	subprocess	trace	wsgiref
binascii	copyreg	filecmp	imghdr	msilib (Windows)	poplib	selectors	sunau	traceback	xdrlib
binhex	crypt (Unix)	fileinput	imp	msvcrt (Windows)	pprint	shelve	symbol	tracemalloc	xml
bisect	csv	fnmatch	importlib	multiprocessing	profile	shlex	symtable	tty (Unix)	xmlrpc
builtins	ctypes	fractions	inspect	netrc	pstats	shutil	sys	turtle	zipapp
bz2	curses (Unix)	ftplib	io	nis (Unix)	pty (Linux)	signal	sysconfig	turtledemo	zipfile
calendar	dataclasses	functools	ipaddress	nntplib	pwd (Unix)	site	syslog (Unix)	types	zipimport
cgi	datetime	gc	itertools	numbers	pyclbr	smtpd	tabnanny	typing	zlib
cgitb	dbm	getopt	json	operator	pydoc	smtplib	tarfile	unicodedata	zoneinfo

Gli ambienti di lavoro



- Ambienti di sviluppo **tradizionali** (IDE)
 - IDLE, PyCharm, Visual Studio Code, Eclipse PyDev, ...
- Ambienti di sviluppo **on-line**
 - Repl.it, PythonAnywhere, Python Tutor
- Ambienti per il calcolo **interattivo**
 - Spyder, IPython
- **Notebook** Computazionali
 - Jupyter, JupyterLab, Google Colab
- Ambienti per **l'apprendimento**
 - Mu, Thonny, Wing

L'IDE di Visual Studio Code

The image shows a screenshot of the Visual Studio Code IDE interface, divided into several panels. The left sidebar contains the Explorer view showing a project structure with folders like 'Settimana08' through 'Settimana14' and files like 'database.txt' and 'dati_aeroporto_torino.csv'. A label 'File di progetto' points to this sidebar. The main editor area displays a Python file named 'soluzione.py' with code for reading data from a CSV file. A label 'Sorgente del programma' points to the code editor. Below the code editor, the 'Exception has occurred: FileNotFoundError' message is visible, indicating an error. A label 'Errori' points to this error message. The bottom panel shows the 'TERMINAL' view with PowerShell output, including the command to activate the Python environment. A label 'Output' points to this terminal view. Other panels visible include 'VARIABLES', 'WATCH', 'CALL STACK', and 'BREAKPOINTS'.

File di progetto

Sorgente del programma

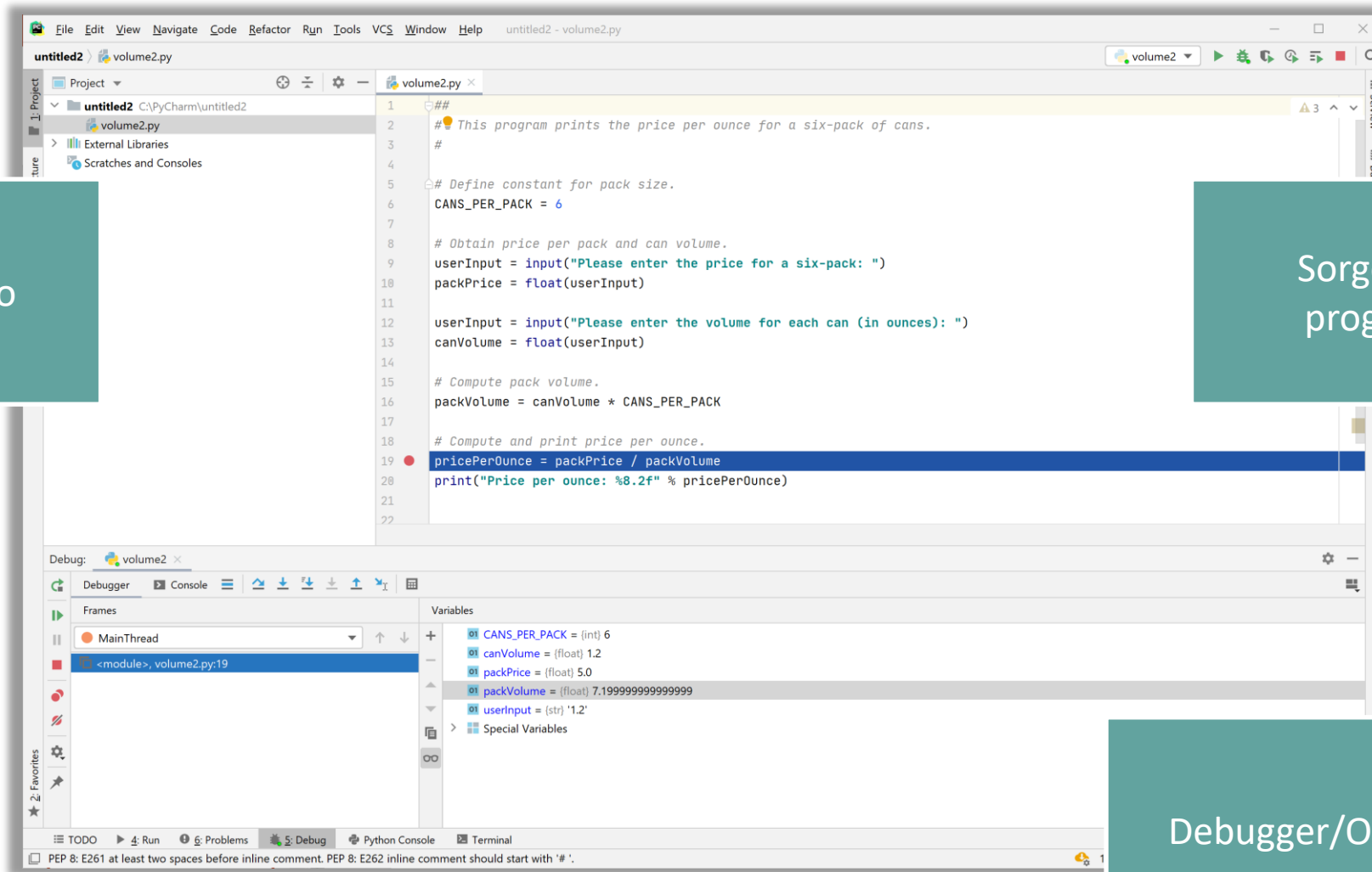
Debugger

Errori

Output

L'IDE di PyCharm

File di progetto

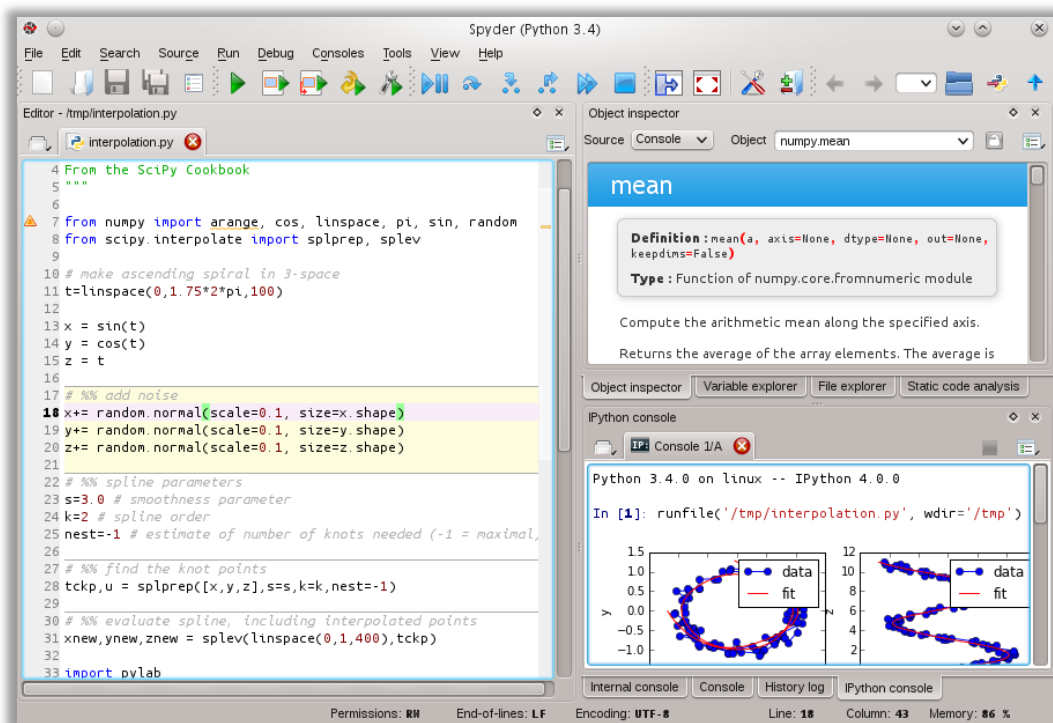


Sorgente del programma

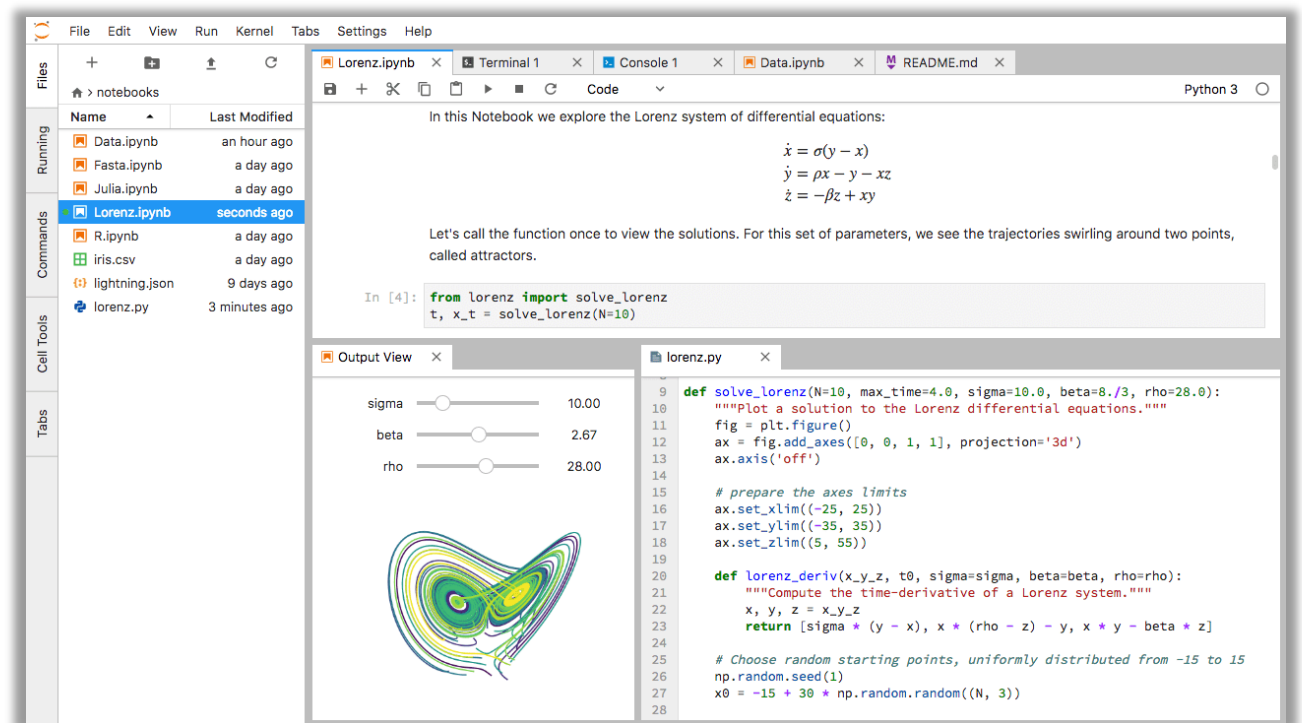
Debugger/Output

Ambienti scientifici interattivi

SPYDER



JUPYTERLAB (ANCHE ON-LINE), GOOGLE COLAB



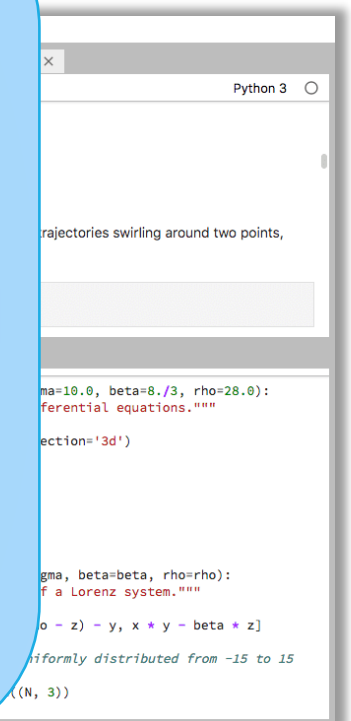
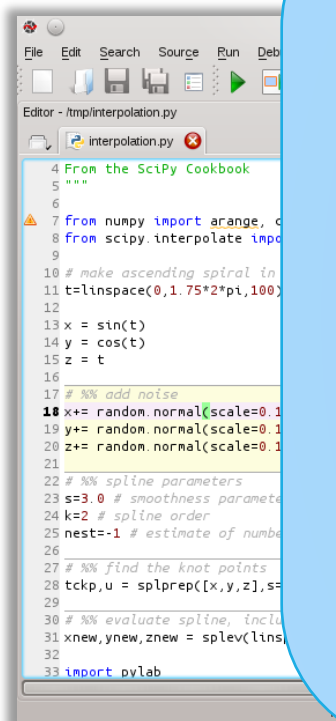
Ambienti scientifici interattivi

SPYDER

JUPYTERLAB (ANCHE ON-LINE), GOOGLE COLAB

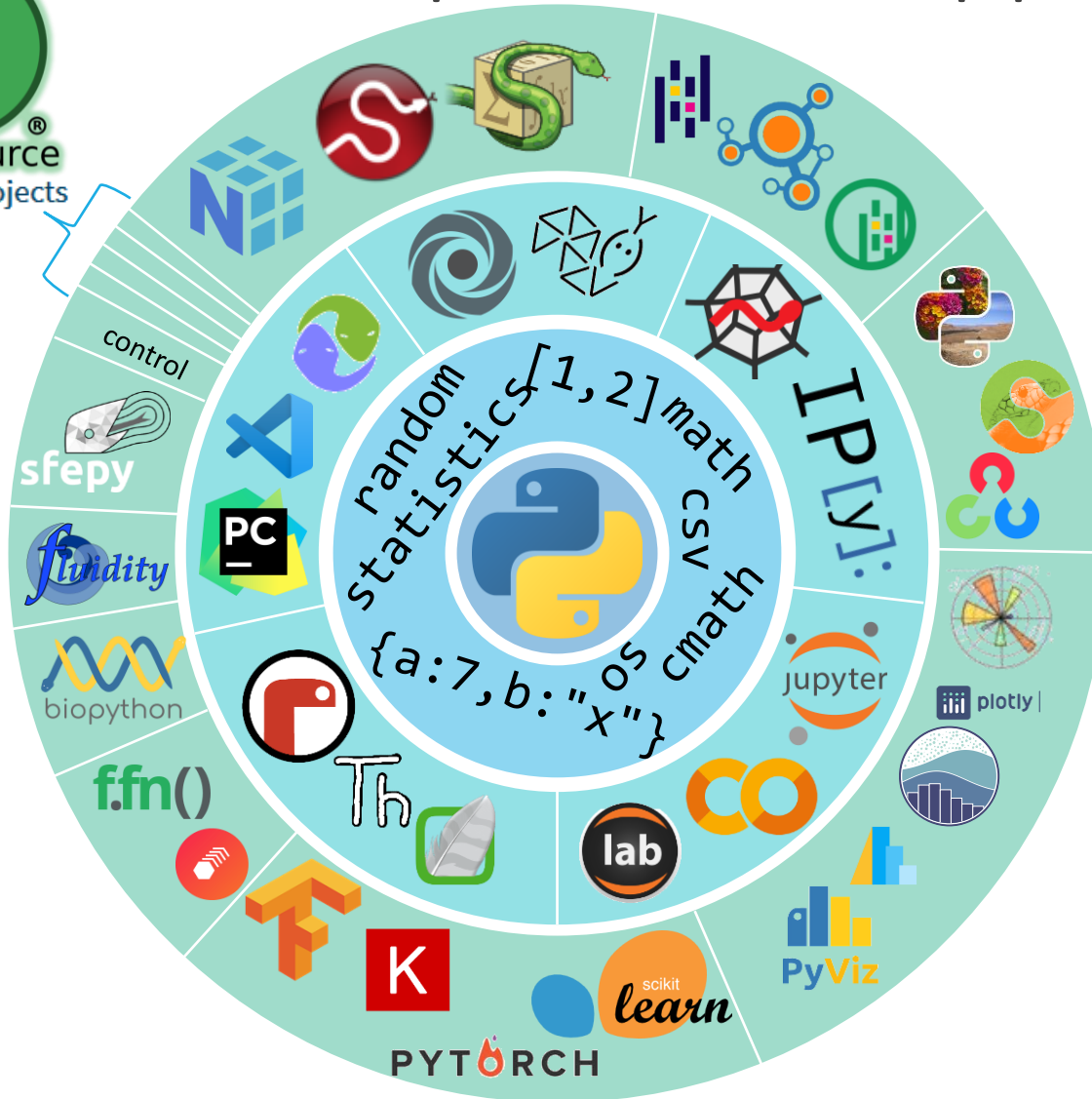
Nuove possibilità...

- Pubblicare on-line esercizi «interattivi» sotto forma di notebook
- Redigere le prime versioni di un articolo, inframmezzando il testo alle formule, con il [ri-]calcolo automatico di risultati e grafici

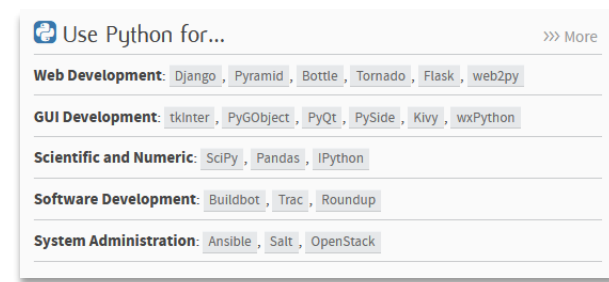


Librerie per ambiti applicativi


open source
304,797 projects



- Scientific computation
 - NumPy, SciPy, SymPy
- Data Analysis, Algoritmi, Grafi
 - Pandas, networkx, GeoPandas
- Image Processing
 - Pillow, scikit-image, OpenCV
- Visualization
 - Pyviz, matplotlib, plotly, seaborn, altair
- Machine Learning
 - Scikit-learn, tensorflow, pytorch, keras
- Fintech
 - f.fn, zipline, pyalgotrade
- Biology and Genome
 - Biopython
- Fluid Dynamics
 - Fluidity
- Finite Elements
 - Sfepy
- Control systems



Singoli moduli



*Toolkit completo
per data science*

Calcolo scientifico



- NumPy
 - Array, vettori, algebra lineare



- SciPy
 - Package specializzati su diversi ambiti scientifici



- SymPy
 - Calcolo simbolico



- Pandas
 - Analisi e manipolazione dati

Subpackage

cluster

constants

fftpack

integrate

interpolate

io

linalg

ndimage

odr

optimize

signal

sparse

spatial

special

stats

Description

Clustering algorithms

Physical and mathematical constants

Fast Fourier Transform routines

Integration and ordinary differential equation solvers

Interpolation and smoothing splines

Input and Output

Linear algebra

N-dimensional image processing

Orthogonal distance regression

Optimization and root-finding routines

Signal processing

Sparse matrices and associated routines

Spatial data structures and algorithms

Special functions

Statistical distributions and functions

Calcolo scientifico



- NumPy
 - Array, vettori, algebra lineare



- SciPy
 - Package specializzati su diversi ambiti scientifici



- SymPy
 - Calcolo simbolico



- Pandas
 - Analisi e manipolazione dati

Features

✓ Core capabilities

✓ Polynomials

✓ Calculus

✓ Solving equations

✓ Combinatorics

✓ Discrete math

✓ Matrices

✓ Geometry

✓ Plotting

✓ Physics

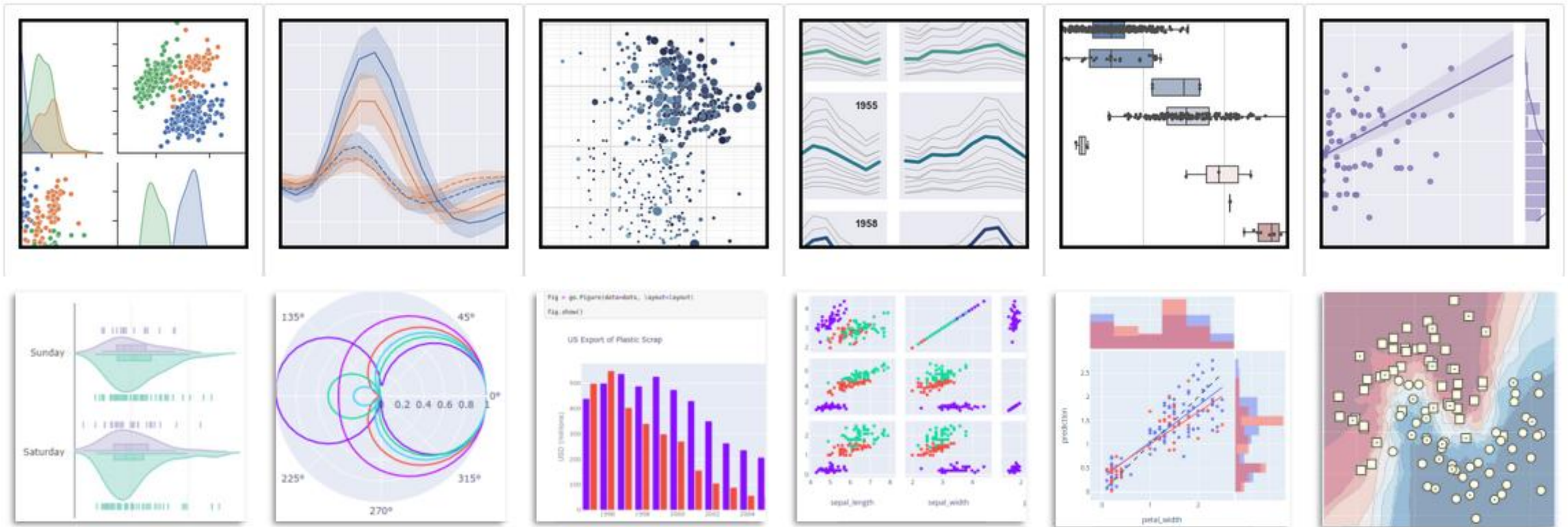
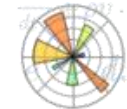
✓ Statistics

✓ Cryptography

✓ Parsing

✓ Printing

Visualizzazione



matplotlib, plotly, seaborn, ...

Esempio: dati ufficiali Covid-19 in real-time

```
import pandas as pd
import seaborn as sns
sns.set_style("whitegrid")

# Leggi dati aggiornati
covid = pd.read_json(
    path_or_buf='https://raw.githubusercontent.com/pcm-dpc/COVID-19/master/dati-json/dpc-covid19-ita-andamento-nazionale.json',
    convert_dates=['data'])

covid.set_index('data', inplace=True)

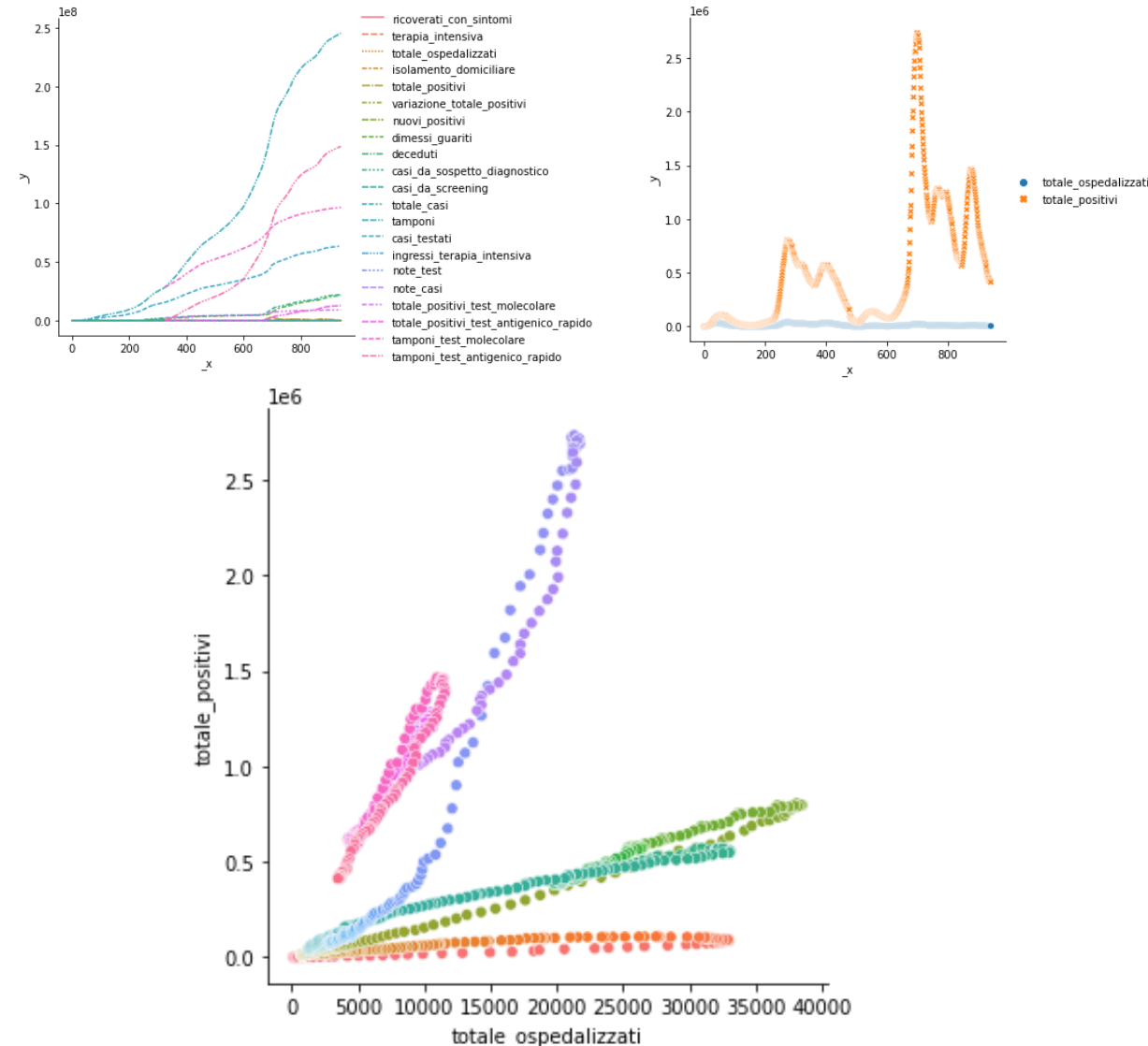
sns.relplot(data=covid, kind='line')

dati_utili = covid[['totale_ospedalizzati', 'totale_positivi']]

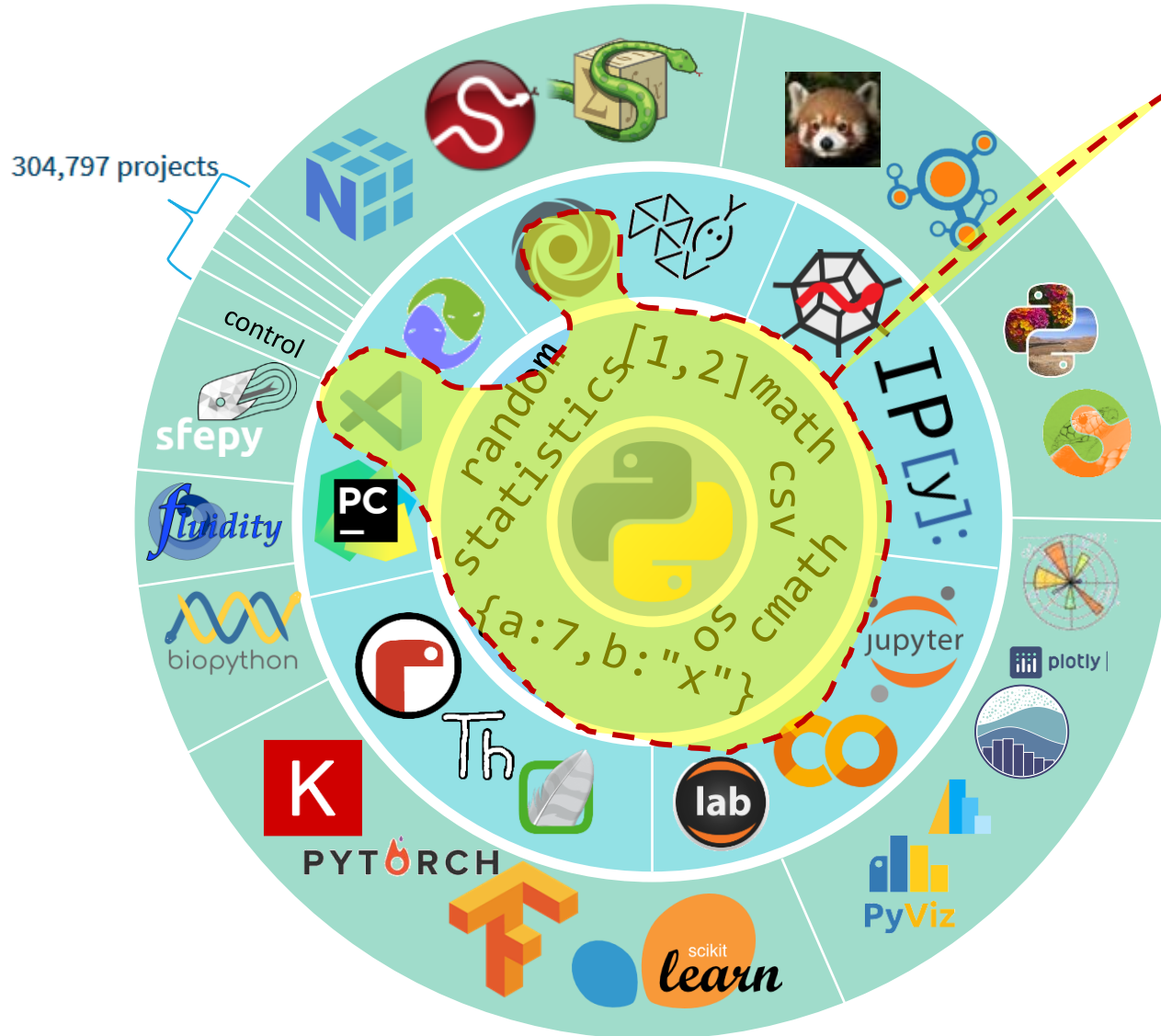
sns.relplot(data=dati_utili, kind='line')

sns.relplot(data=dati_utili, kind='scatter',
            x='totale_ospedalizzati', y='totale_positivi', hue='data',
            legend=False)
```

Try me on Google Colab



Dove arriviamo nel corso del primo anno?



■ Programma del corso di Informatica

- Conoscenza di base del linguaggio
- Familiarità con gli ambienti di sviluppo più semplici
- Capacità di analizzare un problema e formulare un algoritmo

■ Le specializzazioni sulle varie aree non rientrano nel programma di Informatica

- Possibile costruire negli insegnamenti successivi
- Partendo da una base consolidata

Licenza d'uso

- Queste diapositive sono distribuite con licenza Creative Commons “Attribuzione - Non commerciale - Condividi allo stesso modo 2.5 Italia (CC BY-NC-SA 2.5)”
- Sei libero:
 - di riprodurre, distribuire, comunicare al pubblico, esporre in pubblico, rappresentare, eseguire e recitare quest'opera
 - di modificare quest'opera
- Alle seguenti condizioni:
 - **Attribuzione** — Devi attribuire la paternità dell'opera agli autori originali e in modo tale da non suggerire che essi avallino te o il modo in cui tu usi l'opera.
 - **Non commerciale** — Non puoi usare quest'opera per fini commerciali.
 - **Condividi allo stesso modo** — Se alteri o trasformi quest'opera, o se la usi per crearne un'altra, puoi distribuire l'opera risultante solo con una licenza identica o equivalente a questa.
- <http://creativecommons.org/licenses/by-nc-sa/2.5/it/>

